Latihan Concurrency Control pt. 4

1

Diberikan urutan kedatangan instruksi transaksi T1, T2, dan T3 ke DBMS berikut. Transaksi dimulai tepat sebelum instruksi pertama pada transaksi tersebut. Keterangan: R(*Q*) adalah *read* data *Q* dan W(*Q*) adalah *write* pada data *Q* untuk transaksi terkait.

T1	T2	T3
R(A)		
	R(A)	
		R(B)
R(B)		
		W(C)
	W(C)	
R(C)		
commit		
	R(D)	
		W(B)
		commit
	W(D)	
	commit	

Tuliskan langkah-langkah untuk mengeksekusi jadwal di atas sampai semua transaksi tuntas dieksekusi dengan ***multiversion timestamp ordering protocol*** dan jelaskan setiap tahapan eksekusi, termasuk versi data yang dihasilkan. Sebutkan dan jelaskan apa yang terjadi pada tiap transaksi: apakah *abort* atau *commit*. Jika terjadi *abort*, jelaskan bagaimana proses *rollback* dilakukan. Jika dibutuhkan, *timestamp* (Tx,Ty,Tz) = (1,2,3).

Asumsikan: Jika terjadi *abort* yang menyebabkan sebuah transaksi di-*rollback*, DBMS akan memprioritaskan eksekusi ulang dari instruksi transaksi tersebut hingga bagian instruksi yang menyebabkan *abort*.

Jawaban

TS(T1) = 1; TS(T2) = 2; TS(T3) = 3

Versi awal:
<A0, 0, 0>
<B0, 0, 0>
<C0, 0, 0>
<D0, 0, 0>

Langkah eksekusi:

No	Instruksi	Terpilih	Pemeriksaan	Hasil
1	R1(A)	A0	RTS 0 < 1, RTS naik	<A0,1,0>
2	R2(A)	A0	RTS 1<2, RTS naik	<A0,2,0>
3	R3(B)	B0	RTS 0<3, RTS naik	<B0,3,0>
4	R1(B)	B0	RTS 3 >= 1, tidak berubah	<B0,3,0>
5	W3(C)	C0	3 < RTS(C0)=0 → False. 3 != WTS 0, jadi versi baru	<C3,3,3> dibuat
6	W2(C)	C0 (C3 punya	2<RTS(C0)=0 → False. new version	<C2,2,2> dibuat

		WTS 3>2)		
7	R1(C)	C0 (WTS terbesar <=1)	RTS 0<1, RTS naik	<C0,1,0>
8	commit T1	n/a	n/a	T1 commit
9	R2(D)	D0	RTS 0<2, RTS naik	<D0,2,0>
10	W3(B)	B0	3<RTS(B0)=3 → False. new version	<B3,3,3> dibuat
11	commit T3	n/a	n/a	T3 commit
12	W2(D)	D0	2<RTS(D0)=2 → False. new version	<D2,2,2>
13	commit T2	n/a	n/a	T2 commit

2

Diberikan urutan kedatangan instruksi transaksi T1, T2, dan T3 ke DBMS berikut. Transaksi dimulai tepat sebelum instruksi pertama pada transaksi tersebut. Keterangan: R(*Q*) adalah *read* data *Q* dan W(*Q*) adalah *write* pada data *Q* untuk transaksi terkait.

T1	T2	T3
R(A)		
	R(A)	
		R(B)
R(B)		
		W(C)
	W(C)	
R(C)		
commit		
	R(D)	
		W(B)
		commit
	W(D)	
	commit	

Tuliskan langkah-langkah untuk mengeksekusi jadwal di atas sampai semua transaksi tuntas dieksekusi dengan ***multiversion two-phase locking protocol*** dan jelaskan setiap tahapan eksekusi, termasuk versi data yang dihasilkan.

Jawaban

Versi awal: A0, B0, C0, D0 (WTS = 0).

Langkah eksekusi:

No	Instruksi	Proses

1	R1(A)	T1 mulai, TS(T1) = ts-counter = 0. Baca A0 tanpa lock
2	R2(A)	T2 ambil S(A), baca versi terbaru: A0
3	R3(B)	T3 ambil S(B), baca B0
4	R1(B)	T1 (TS=0) baca B0 tanpa lock
5	W3(C)	T3 ambil X(C), buat versi C∞
6	W2(C)	T2 butuh X(C) yang dipegang T3, jadi T2 menunggu T3
7	R1(C)	T1 (TS=0) baca C0. C∞ tidak terlihat karena WTS ∞ > 0, jadi T1 tidak menunggu
8	commit T1	Read-only, tidak ada lock yang dilepas dan ts-counter tidak berubah
9	R2(D)	T2 masih menunggu di W2(C), sehingga instruksi ini tertahan
10	W3(B)	T3 upgrade S(B) → X(B). Berhasil karena tidak ada pemegang S(B) lain (T1 tanpa lock). Buat B∞
11	commit T3	TS(T3) = 0 + 1 = 1. C∞ → C1, B∞ → B1. ts-counter = 1. Lepas X(C) dan X(B)
12	W2(D)	T2 mendapat X(C), buat C∞. Lalu R2(D): S(D), baca D0. W2(D): upgrade ke X(D), buat D∞
13	commit T2	TS(T2) = 1 + 1 = 2. C∞ → C2, D∞ → D2. ts-counter = 2. Lepas S(A), X(C), X(D)

3

Diberikan urutan kedatangan instruksi transaksi T1, T2, dan T3 ke DBMS berikut. Transaksi dimulai tepat sebelum instruksi pertama pada transaksi tersebut.
Keterangan: R(*Q*) adalah *read* data *Q* dan W(*Q*) adalah *write* pada data *Q* untuk transaksi terkait.
Asumsikan nilai dari tiap data di awal A=10; B=20; C=0; D=30

T1	T2	T3
R(A)		
	R(A)	
		R(B)
R(B)		
		C:=50
		W(C)
	C:=35	
	W(C)	
R(C)		
commit		
	R(D)	
		B:=B-0.1*C
		W(B)
		commit
	D:=0.2*D+A	
	W(D)	
	commit	

Tuliskan langkah-langkah untuk mengeksekusi jadwal di atas sampai semua transaksi tuntas dieksekusi dengan ***snapshot isolation*** (dengan *first-committer wins*) dan jelaskan setiap tahapan eksekusi, termasuk versi data yang dihasilkan dan nilai dari tiap data. Sebutkan dan jelaskan apa yang terjadi pada tiap

Jawaban

Nilai awal: A0=10, B0=20, C0=0, D0=30.

Langkah eksekusi:

No	Instruksi	Proses	Hasil
1	R1(A)	Baca snapshot T1	A0=10
2	R2(A)	Baca snapshot T2	A0=10
3	R3(B)	Baca snapshot T3	B0 = 20
4	R1(B)	Baca snapshot T1	B0 = 20
5	W3(C)	Ditulis di salinan privat T3	C3 = 50
6	W2(C)	Ditulis di salinan privat T2	C2 = 35
7	R1(C)	Tulisan T2 dan T3 belum committed, tidak terlihat	C0 = 0
8	commit T1	Write set kosong, tidak ada konflik	commit
9	R2(D)	Baca snapshot T2	D0 = 30
10	W3(B)	B=20, C=50	B3 = 20 − 0.1*50 = 15

11	commit T3	Write set {C, B}. Belum ada transaksi konkuren yang commit dan menulis C atau B (T1 hanya membaca)	commit: C3 = 50, B3 = 15
12	W2(D)	0.2*30 + 10 (A dari snapshot T2)	D2 = 16
13	commit T2	Write set {C, D}. T3 konkuren dengan T2 dan sudah commit lebih dulu dengan menulis C, sehingga melanggar first-committer-wins	Abort

T1 commit, T3 commit, T2 abort karena write set T2 (C dan D) beririsan dengan T3 © yang konkuren dan sudah commit terlebih dahulu. Oleh karena itu, perlu rollback T2. Semua penulisan T2 (C2 = 35 dan D2 = 16) hanya ada di salinan privat dan belum masuk ke database sehingga rollback cukup membuang kedua versi tersebut.Maka dari itu, versi dan nilai data akhir adalah:

Data	Versi	Nilai
A	A0	10
B	B0, B3	20 → 15
C	C0, C3	0 → 50
D	D0	30 (D2 dibuang karena T2 abort)